

본의 사o 잠언 m라터데 라서자 장용 에이제 장설자아 소주의 장예의 심자o 잠언 m

LW강해WUo간결목  
L  
La결국개  
: LB?BD

W강해WU5r nn W강해WU 0 강과무결결결결n강해 6  
W강해WU i fTk 4W강해WU5r nn 6W강해WU  
6 4 6 6 4

이 B

L  
L WU 4  
i fTk L  
L WU  
L WU  
LW강해WU

이 B o 잠언 m

L  
구개1 양목목공  
i fTk  
  
i fTkL  
nl f  
  
L  
4 4

이 B o 잠언 m라터데

L  
W qUf[wL  
]fi TowL

```
-- L2 거리 (유클리드 거리)
SELECT id, text,
       array_distance(vector, [1.2, 3.4, ...]) AS distance
FROM embeddings
ORDER BY distance
LIMIT 10;

-- 코사인 유사도
SELECT id, text,
       array_cosine_similarity(vector, [1.2, 3.4, ...]) AS similarity
FROM embeddings
ORDER BY similarity DESC
LIMIT 10;

-- 내적
SELECT id, text,
       array_dot_product(vector, [1.2, 3.4, ...]) AS dot
FROM embeddings
ORDER BY dot DESC
LIMIT 10;
```

bhns Ob 갈구갈개갈고 갈꺾갈개갈곡갈교 과과과 L

L  
가림과 강행  
곡Lbhrs  
간편 0 1

```

graph TD
    A[SQL 쿼리] --> B[벡터 연산 플래너]
    B --> C[HNSW 인덱스 (선택적)]
    C --> D[컬럼 저장소 (벡터 데이터)]

```

↓  
메모리/디스크

L

ng W

bhns L  
bhns

L

```
SELECT id, text, category,  
       array_distance(vector, query_vec) AS dist  
FROM embeddings  
WHERE category = 'news'      -- 스칼라 필터  
      AND date >= '2024-01-01' -- 추가 필터  
ORDER BY dist  
LIMIT 10;
```

L

A6                    0            1  
B6  
C6

이 B

관계 기반

```
-- 임베딩된 뉴스 기사에서 특정 주제와 유사하면서  
-- 특정 기간의 기사 분석  
SELECT category, COUNT(*) as count,  
       AVG(array_distance(embedding, topic_vec)) as avg_similarity  
FROM articles  
WHERE published_date >= '2024-01-01'  
GROUP BY category  
HAVING AVG(array_distance(embedding, topic_vec)) < 0.5  
ORDER BY avg_similarity;
```

I x

```
import duckdb  
  
conn = duckdb.connect()  
  
# 벡터 데이터 로드  
conn.execute("""  
    CREATE TABLE documents AS  
    SELECT doc_id, content, embedding  
    FROM read_csv('docs.csv')  
""")  
  
# 검색 쿼리  
query_vec = [0.1, 0.2, 0.3, ...]  
results = conn.execute(f"""  
    SELECT doc_id, content,  
           array_distance(embedding, {query_vec}) as distance  
    FROM documents  
    ORDER BY distance  
    LIMIT 10  
""").fetchall()
```

r 닻

```
-- 벡터 생성 및 저장
CREATE TABLE embeddings AS
SELECT id,
       text,
       ai_embed(text) as vector -- 커스텀 임베딩 함수
FROM raw_documents
WHERE valid = true;

-- 벡터 인덱싱
CREATE INDEX ON embeddings USING hnsw (vector);

-- 배치 검색
SELECT DISTINCT doc_a, doc_b
FROM (
  SELECT a.id as doc_a, b.id as doc_b,
         array_distance(a.vector, b.vector) as sim
  FROM embeddings a
  JOIN embeddings b ON a.category = b.category
) t
WHERE sim < 0.1
LIMIT 1000;
```

이 B



이 B

|        |   |         |       |           |  |        |   |
|--------|---|---------|-------|-----------|--|--------|---|
| [ 갈과과무 |   |         |       | 6W강U5r nn |  | [ 갈과과무 | L |
| A6     |   | L[ 갈과과무 | i fTk | 4W강U5r nn |  |        |   |
| B6     | 5 | L[ 갈과과무 |       | W강U5r nn  |  |        |   |
| C6     |   | L[ 갈과과무 |       | W강U5r nn  |  |        |   |

A6 L Q Q

B6bhns L곡 검 4 Q

C6 LW구배W Q Q

D6 L bhns Q

E6 L Q 0A??4??31

F6 LW구배W5 nn k결간행문4결문 WU 5 Q